

1. What is a Class?

A **class** is like a blueprint for creating objects. It defines the attributes (variables) and behaviors (methods) that the objects created from the class will have.

- **Attributes:** Characteristics or properties that an object of the class will have.
- **Methods:** Functions that define the behaviors of the class (what the objects can do).

Think of a **class** as a template for creating objects. For example, you could have a class **Car**, and every car created from that class would share the same properties (like color, model, and speed) and behaviors (like drive, stop).

2. What is an Object?

An **object** is an instance of a class. When you create an object, you are creating an actual entity that is based on the class blueprint.

Each object can have its own specific attributes (e.g., one car may be red, while another is blue) but will share the same methods (e.g., all cars can drive or stop).

For example:

- Class: **Car**
- Object: **car1**, **car2**, etc.

3. Creating a Class in Python

In Python, you create a class using the **class** keyword. Here's a simple example:

Example of a Class:

```
class Car:
    # Constructor (initializer)
    def __init__(self, brand, model, year):
        self.brand = brand # Attribute for car brand
        self.model = model # Attribute for car model
        self.year = year   # Attribute for car year

    # Method to display car details
    def display_details(self):
        print(f"Car Brand: {self.brand}")
        print(f"Car Model: {self.model}")
        print(f"Car Year: {self.year}")

    # Method to simulate starting the car
```

```
def start_engine(self):
    print(f"The {self.model} engine is now running!")

# Method to simulate stopping the car
def stop_engine(self):
    print(f"The {self.model} engine has been stopped!")
```

Explanation:

- **`__init__` method**: This is the constructor. It is called when an object of the class is created and initializes the object with the provided attributes (brand, model, year).
- **`display_details` method**: A method that prints the details of the car.
- **`start_engine` and `stop_engine` methods**: Methods that represent actions or behaviors the car object can perform.

4. Creating Objects from the Class

Once you have defined the class, you can create objects (instances) of that class by calling the class like a function.

Example of Creating Objects:

```
# Create instances (objects) of the Car class
car1 = Car("Toyota", "Corolla", 2020)
car2 = Car("Honda", "Civic", 2022)

# Accessing methods using the objects
car1.display_details() # Display details of car1
car1.start_engine()    # Start the engine of car1

car2.display_details() # Display details of car2
car2.stop_engine()     # Stop the engine of car2
```

Explanation:

- `car1` and `car2` are **objects** of the `Car` class.
- We pass specific values (brand, model, and year) to the constructor (`__init__` method) when creating each object.
- Each object can call the methods defined in the class, and the methods act on the object's attributes.

. Attributes and Methods of the Object

- **Attributes:** When you create an object of the class, each object can hold its own values for the attributes. For example, `car1` and `car2` have different `brand`, `model`, and `year` attributes

```
print(car1.brand) # Output: Toyota
print(car2.model) # Output: Civic
```

Methods: The methods defined in the class can be called using the object. Methods operate on the attributes of the object

Example with Output

Let's walk through the output of the above example:

```
car1 = Car("Toyota", "Corolla", 2020)
car2 = Car("Honda", "Civic", 2022)
```

```
car1.display_details() # Output: Car Brand: Toyota, Car Model: Corolla, Car Year: 2020
car1.start_engine()    # Output: The Corolla engine is now running!
```

```
car2.display_details() # Output: Car Brand: Honda, Car Model: Civic, Car Year: 2022
car2.stop_engine()     # Output: The Civic engine has been stopped!
```

`car1.display_details()` will print the details of `car1`.

`car2.display_details()` will print the details of `car2`.

`car1.start_engine()` simulates starting the engine of the `car1`.

`car2.stop_engine()` simulates stopping the engine of the `car2`.

Key Points to Remember

- A **class** is a blueprint for creating objects.
- An **object** is an instance of a class.
- Classes have **attributes** (characteristics) and **methods** (behaviors).
- Objects are created using the class and can have their own specific attribute values.
- The **constructor** (`__init__` method) initializes the object with specific values when it is created

Summary

- **Class:** A blueprint or template that defines attributes and methods.
- **Object:** An instance of a class with its own set of attributes and can call methods to perform actions.
- **Attributes:** Variables that belong to an object or class.
- **Methods:** Functions defined inside a class that operate on the object's data.

In object-oriented programming, classes and objects allow you to model real-world entities and define their behaviors and properties in an organized manner.